**GEBZE TECHNICAL UNIVERSITY**

Computer Engineering Department

CSE 396 Module Documentation

# Çılgın Kurye

*Module M7: ESP32 Software and Communication*

**Instructor:** Erkan Zergeroğlu **Company:** Board&Byte **Group:** Group-8 **Date:** 6 November 2026

| Name | Student No. | Name | Student No. |
|---|---|---|---|
| Tuğberk Akbulut | 230104004070 | Hasan Alperen Elmas | 1801042604 |
| Hasan Hüseyin Özdemir | 230104004048 | Muhib Taha Boy | 230104004165 |

## 1. Purpose

M7 is the software of the ESP32 inside the LED board, and the link between the board and the main computer. It gets the game state over Wi-Fi and gives it to the LED engine (M8). It reads the motion sensor and finds the strength and the place of a hit on the board. It drives the vibration motors. It also reports the health of the board and the sensor signal to the admin.

| | |
|---|---|
| Runs on | ESP32 in the LED board: core 0 (the LED engine M8 runs on core 1) |
| Technology | C++ with the Arduino framework for ESP32 and FreeRTOS tasks; TCP and UDP over Wi-Fi with MessagePack; 6-axis motion sensor (accelerometer and gyroscope); 4 vibration motors with transistor drivers |
| Requirements | R7 R8 R10 R11 |

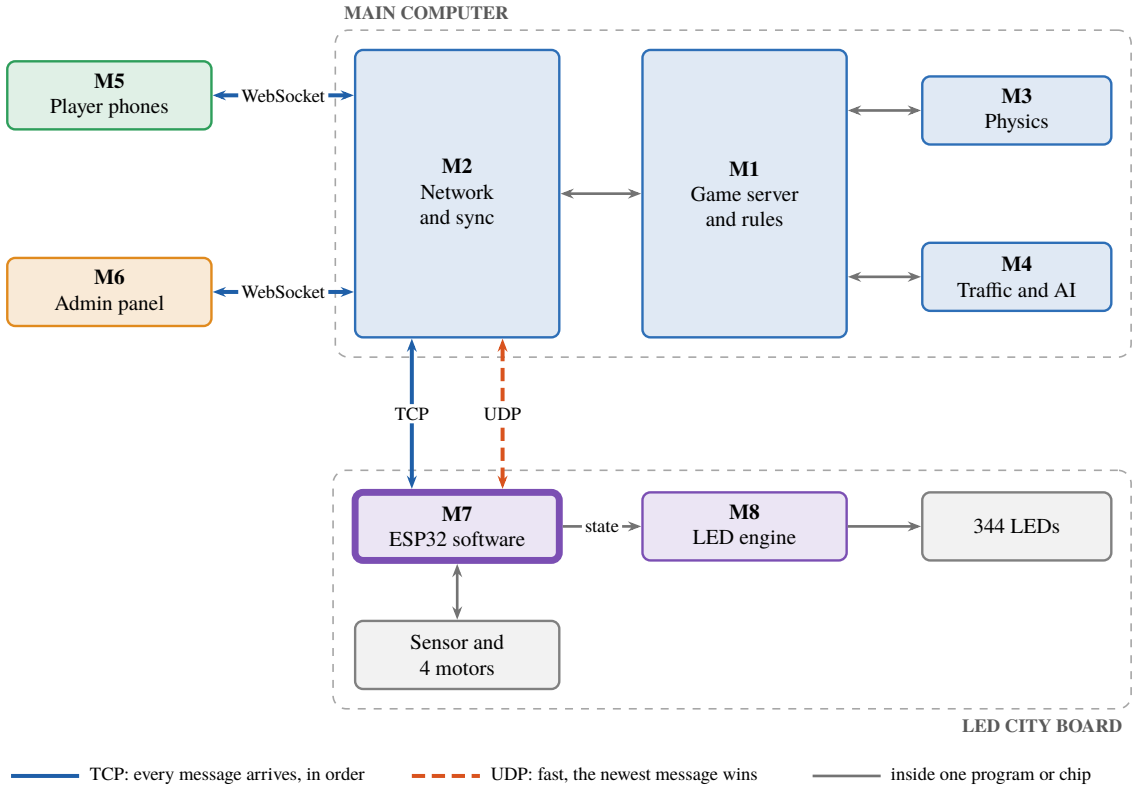



**Figure 1:** Where M7 sits in the system. M7 talks to the server on two channels: TCP and UDP.

## 2. Requirements covered

| ID | Requirement | What M7 does for it |
|---|---|---|
| R7 | Earthquake | Finds hand hits and motor shaking with the sensor; strength 0–10 and a 3 × 3 zone; drives the motors for the admin button |
| R8 | Physical LED board | Brings the game state to the board over Wi-Fi; one ESP32 for the LEDs, the sensor and the motors |
| R10 | Network and measurement | Board delay measurement; sensor samples for the seismic graph; comes back after a lost link |

| ID | Requirement | What M7 does for it |
|---|---|---|
| R11 | Independent development | PC emulators for both sides of the link |

# 3. Inputs and outputs

| Dir. | Data | From / to | Content |
|---|---|---|---|
| In | Board state | M2, UDP | 10 to 30 per second: phase, players (team, tone, position, flags), delivery points, obstacles, earthquake |
| In | Config | M2, TCP | Player colours, brightness 0.1–1, LED current limit |
| In | Vibrate | M2, TCP | Motors to use, strength 0–1, duration of at most 5 s |
| In | Sensor | Hardware | 1000 samples per second |
| Out | Newest state | M8 | Latest board state, config and link state, on the same chip |
| Out | Earthquake event | M2, TCP, to M1 and M6 | Source (hand or motors), strength 0–10, centre, zone 0–8, confidence |
| Out | Sensor samples | M2, UDP, to M6 | 100 per second for the seismic graph, sent in groups of 10 |
| Out | Status | M2, TCP, to M6 | Once per second: frame rate, Wi-Fi signal, LED current, errors |

# 4. Main components

| Component | What it does | Related modules |
|---|---|---|
| Network link | Joins Wi-Fi, finds the server, keeps the TCP channel and the UDP stream, connects again after a loss | M2 |
| Message codec | MessagePack encode and decode; checks the version and the order numbers; drops old or broken messages | M2 |
| State hand-over | Two-slot buffer: the LED engine always takes the newest full state and never waits | M8 |
| Hit detector | Finds hits in the sensor signal; computes the strength and the zone | M1, M6 |
| Motor controller | Runs vibrate commands with a soft start and safety limits | M1, M6 |
| Status and settings | Status reports and delay replies; Wi-Fi and calibration settings in flash, changed over USB | M6 |

# 5. How it works

**Two channels.** The board keeps one TCP channel and one UDP stream to the server. Messages that must arrive go on TCP: the config, the vibrate commands, the earthquake events and the status report. Data that comes again and again goes on UDP: the board state, the sensor samples and the delay probes. If one UDP packet is lost, the next one replaces it, so the LEDs never wait. Every message has an order number, so the board can drop a UDP packet that comes late and out of order.

**Two cores.** The ESP32 has two cores. Core 0 runs all M7 work: Wi-Fi, the network tasks, the sensor, the hit detector and the motors. Core 1 runs only the LED engine of M8, 60 frames per second. So Wi-Fi work never breaks the LED timing. The two sides share the state through a two-slot buffer. M7 always writes into the free slot. When the slot is full, M7 switches one index in one step. The LED engine reads the slot that the index shows, so it always gets a complete state and it never waits for a lock.

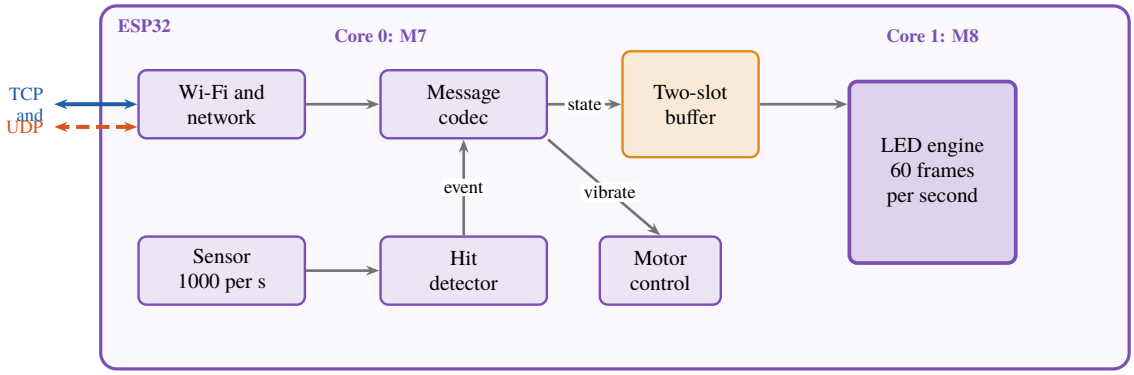



**Figure 2:** Work on the two cores of the ESP32. The buffer is the only shared part.

**What is inside the board.** The figure below is a side cut of the board. The motion sensor is fixed at the centre, under the plywood, so it feels a hit from every side. The four vibration motors are under the four corners. They have their own power line, so they do not make the LED voltage drop. The ESP32, the level shifter and the power supply sit under the board too.

**Finding a hit.** The sensor measures the motion 1000 times per second. First a filter takes away gravity and slow tilting, so

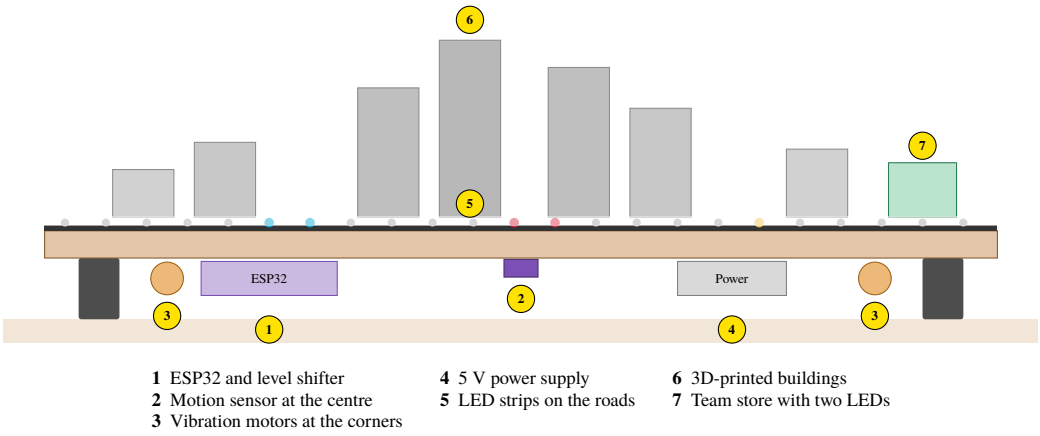



**Figure 3:** Side cut of the board with the parts that M7 uses.

only the shaking is left. The detector also learns the normal noise of the room. The start threshold is the mean noise plus six times its spread, so it goes up on a noisy table. When the signal passes the threshold, a hit starts. The detector takes a 150 ms window from this moment. The energy E in the window gives the strength with a log scale, like a real earthquake scale: strength = k × log10(E / E0). Here E0 is the energy of the weakest hit we accept, and k is set in calibration with a standard hit. The result is cut to 0–10. The first push of the board shows the direction of the hit, and its size shows how far the hit is from the centre. From this the detector picks one of 9 zones in a 3 × 3 grid and gives a confidence value. After a hit the detector waits 1 second, so one hit makes only one event. The event goes to the server within 200 ms.

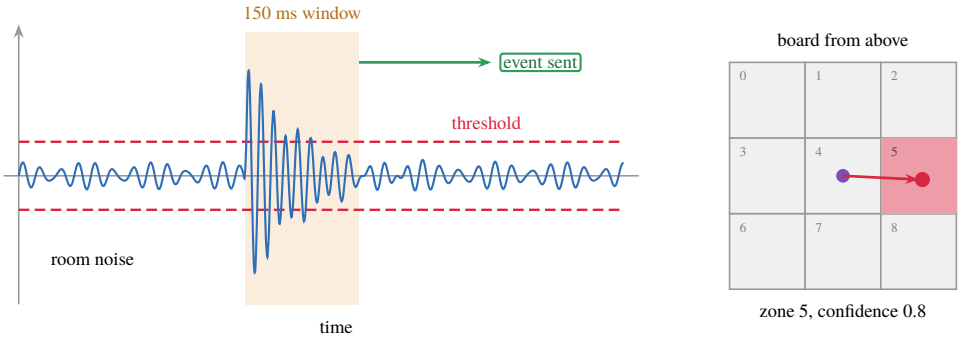



**Figure 4:** Left: the sensor signal of one hand hit. Right: the 3 × 3 zones; the purple dot is the sensor.

**Motors.** When the admin presses the earthquake button, M1 sends a vibrate command on TCP. The command says which motors to use, the strength and the time, at most 5 seconds. M7 drives the motors with PWM through transistors. It starts slowly over 100 ms, so the current does not jump. The shaking is then measured by the same detector as a hand hit, and the event is marked with “motors” as its source. So the earthquake in the game follows the real shaking of the table.

**When the link is lost.** If no state comes for 2 seconds, M7 stops the motors at once and tells M8 to show a “searching” animation. Then it tries to connect again. The goal is to be back within 5 seconds after the router is up. The Wi-Fi name, the password and the calibration values are stored in the ESP32 flash. We can change them over the USB cable, so we do not need to build the code again at a new place.

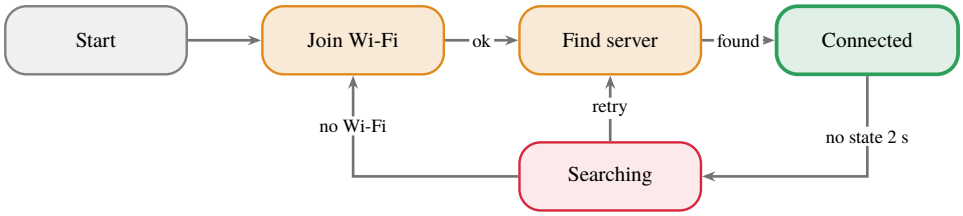



**Figure 5:** Link states of the board. In “Searching” the motors stop and the LEDs show a waiting animation.

## 6. Testing and demonstration

Two PC programs take the place of the neighbours. The server emulator sends test scenarios with delay and loss that we can change. It also shows the earthquake events and the seismic signal. The board emulator draws the board on the screen, so

M1 and M2 can test before the real board is ready. The tests use the shared example messages, broken packets, a 2-hour run, added delay, 20% loss and a router restart. For the sensor we test hit finding and false alarms. Our targets are: at least 95% of calibrated hits are found; at most one false earthquake in 30 minutes of normal play; the correct quarter of the board in at least 95% of hits and the correct zone in 80%; and at most 200 ms from the hit to the event. In the demonstration we hit the board by hand and press the admin button, and we watch the seismic graph and the shock wave on the LEDs.